UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 03

NOMBRE COMPLETO: Lechuga Castillo Shareny Ixchel

Nº de Cuenta: 319004252

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 04

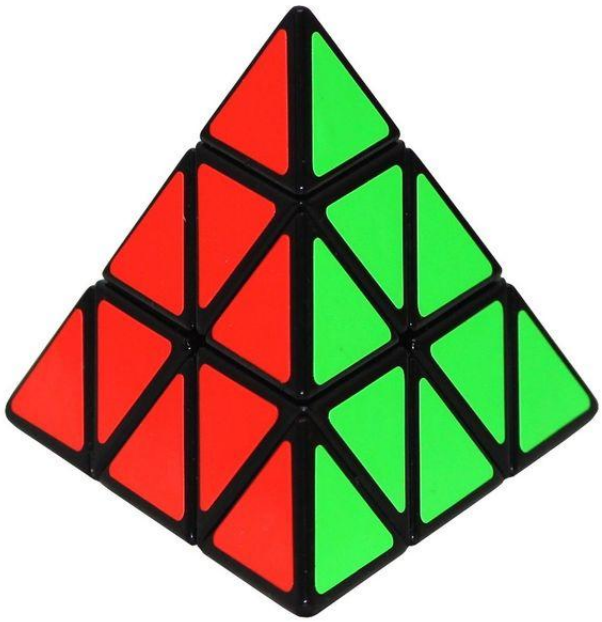
SEMESTRE 2026-1

FECHA DE ENTREGA LÍMITE: 11-09-2025

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Generar una pirámide rubik (pyraminx) de 9 **pirámides** por cara. Cada cara de la pyraminx que se vea de un color diferente y que se vean las separaciones entre instancias (las líneas oscuras son las que permiten diferenciar cada pirámide pequeña)
Agregar en su documento escrito las capturas de pantalla necesarias para que se vean las 4 caras de toda la pyraminx o un video en el cual muestra las 4 caras



Código:

```
42  void CrearPiramideTriangular()
43  {
44      //-------------- BASE PIRAMIDE (NEGRA) ---------------------
45      unsigned int indices_piramide_triangular[] = {
46          0, 1, 2,
47          1, 3, 2,
48          3, 0, 2,
49          1, 0, 3
50      };
51
52      float max = 2.0f;
53
54      GLfloat vertices_piramide_triangular[] = {
55          -0.65f * max, -0.55f * max, 0.0f * max,
56          0.65f * max, -0.55f * max, 0.0f * max,
57          0.0f * max, 0.7f * max, -0.36f * max,
58          -0.1f * max, -0.57f * max, -1.1f * max
59      };
60
61      Mesh* obj1 = new Mesh();
62      obj1->CreateMesh(vertices_piramide_triangular, indices_piramide_triangular, 12, 12);
63      meshList.push_back(obj1);
64
65      //-------------- PiIRAMIDES ---------------------
66      unsigned int punta_indices_piramide_triangular[] = {
67          0, 1, 2,
68          1, 3, 2,
69          3, 0, 2,
70          1, 0, 3
71      };
72
73      GLfloat punta_vertices_piramide_triangular[] = {
74          -0.7f, -0.7f, 0.0f,  // V•rtice 0
75          0.7f, -0.7f, 0.0f,   // V•rtice 1
76          0.0f, 0.7f, -0.3f,   // V•rtice 2 Cima
77          0.0f, -0.65f, -1.0f   // V•rtice 3 Base de atr•s
78      };
```

```
    Mesh* punta = new Mesh();
    punta->CreateMesh(punta_vertices_piramide_triangular, punta_indices_piramide_triangular, 12, 12);
    meshList.push_back(punta);
}

void CreateShaders()
{
    Shader *shader1 = new Shader();
    shader1->CreateFromFiles(vShader, fShader);
    shaderList.push_back(*shader1);

    Shader* shader2 = new Shader();
    shader2->CreateFromFiles(vShaderColor, fShader);
    shaderList.push_back(*shader2);
}


int main()
{
    mainWindow = Window(800, 600);
    mainWindow.Initialise();

    CrearPiramideTriangular();
    CreateShaders();

    camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.3f);

    GLuint uniformProjection = 0;
    GLuint uniformModel = 0;
    GLuint uniformView = 0;
    GLuint uniformColor = 0;
    glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);

    while (!mainWindow.getShouldClose())
    {
        GLfloat now = glfwGetTime();
        deltaTime = now - lastTime;
        deltaTime += (now - lastTime) / limitFPS;
        deltaTime += (now - lastTime) / limitFPS;
        lastTime = now;
        glfwPollEvents();
        camera.keyControl(mainWindow.getsKeys(), deltaTime);
        camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());

        glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
        glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

        // Renderizar la base (pirámide negra)
        shaderList[0].useShader();
        uniformModel = shaderList[0].getModelLocation();
        uniformProjection = shaderList[0].getProjectLocation();
        uniformView = shaderList[0].getViewLocation();
        uniformColor = shaderList[0].getColorLocation();

        glm::mat4 model = glm::mat4(1.0);
        model = glm::translate(model, glm::vec3(0.0f, 0.0f, -4.0f));
        glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
        glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
        glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
        glm::vec3 color = glm::vec3(0.0f, 0.0f, 0.0f); // Color negro
        glUniform3fv(uniformColor, 1, glm::value_ptr(color));
        meshList[0]->RenderMesh(); // Renderizar la pirámide negra

    //--------------- COLOR ROJO ------------------------------------------
        // Renderizar la punta
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(0.0f, 0.9f, -4.42f));
            model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));

            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(1.0f, 0.0f, 0.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        // Renderizar segundo piso (1er triangulo de izquierda a derecha)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(-0.37f, 0.10f, -4.2f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(1.0f, 0.0f, 0.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        // Renderizar segundo piso (enmedio rotado)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(0.0f, 0.18f, -4.22f));
            model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));
```

```
		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	// Renderizar segundo piso (3er triangulo de izquierda a derecha)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
		uniformColor = shaderList[0].getColorLocation();

		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(0.37f, 0.10f, -4.2f));
		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	// Renderizar tercer piso (1er triangulo de izquierda a derecha)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
		uniformColor = shaderList[0].getColorLocation();

		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(-0.78f, -0.68f, -4.01f));
		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	//Renderizar el tercer piso (1er rotado)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
		uniformColor = shaderList[0].getColorLocation();

		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(0.39f, -0.6f, -4.04f));
		model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
		model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));

		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	// Renderizar tercer piso (3er triangulo de izquierda a derecha)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
		uniformColor = shaderList[0].getColorLocation();

		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(0.0f, -0.68f, -4.01f));
		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	//Renderizar el tercer piso (2do rotado)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
		uniformColor = shaderList[0].getColorLocation();

		model = glm::mat4(1.0);
		model = glm::translate(model, glm::vec3(-0.39f, -0.6f, -4.04f));
		model = glm::rotate(model, glm::radians(180.0f), glm::vec3(0.0f, 0.0f, 1.0f));
		model = glm::rotate(model, glm::radians(25.0f), glm::vec3(1.0f, 0.0f, 0.0f));

		model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
		glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
		glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
		glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
		color = glm::vec3(1.0f, 0.0f, 0.0f);
		glUniform3fv(uniformColor, 1, glm::value_ptr(color));
		meshList[1]->RenderMesh();

	// Renderizar tercer piso (5to triangulo de izquierda a derecha)
		shaderList[0].useShader();
		uniformModel = shaderList[0].getModelLocation();
		uniformProjection = shaderList[0].getProjectLocation();
		uniformView = shaderList[0].getViewLocation();
```

```
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(0.78f, -0.68f, -4.01f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(1.0f, 0.0f, 0.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        //---------------------- COLOR AZUL ----------------------------------

        // Renderizar la punta
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(0.032f, 0.9f, -4.88f));
            model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
            model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(0.0f, 0.0f, 1.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        // Renderizar 2da fila (primero)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
```

```
            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(-0.07f, 0.1f, -5.34f));
            model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
            model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(0.0f, 0.0f, 1.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        // Renderizar 2da fila (rotado)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(-0.3f, 0.1f, -5.05f)); //-0.3 x

            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            model = glm::rotate(model, glm::radians(180.0f), glm::vec3(1.0f, 0.0f, 0.0f)); //Rotando
            model = glm::rotate(model, glm::radians(59.0f), glm::vec3(0.0f, 1.0f, 0.0f));
            model = glm::rotate(model, glm::radians(42.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::rotate(model, glm::radians(-21.0f), glm::vec3(1.0f, 0.0f, 0.0f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(0.0f, 0.0f, 1.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();
```

```
        // Renderizar 2da fila (segundo)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(-0.37f, 0.1f, -4.67f));
            model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
            model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(0.0f, 0.0f, 1.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
            meshList[1]->RenderMesh();

        // Renderizar 3ra fila (primero)
            shaderList[0].useShader();
            uniformModel = shaderList[0].getModelLocation();
            uniformProjection = shaderList[0].getProjectLocation();
            uniformView = shaderList[0].getViewLocation();
            uniformColor = shaderList[0].getColorLocation();

            model = glm::mat4(1.0);
            model = glm::translate(model, glm::vec3(-0.151f, -0.68f, -5.81f));
            model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 1.0f, 0.0f));
            model = glm::rotate(model, glm::radians(2.0f), glm::vec3(0.0f, 0.0f, 1.0f));
            model = glm::rotate(model, glm::radians(0.5f), glm::vec3(1.0f, 0.0f, 0.0f));
            model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
            glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
            glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
            glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
            color = glm::vec3(0.0f, 0.0f, 1.0f);
            glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
//---------------- COLOR VERDE ----------------------------------------------
    // Renderizar la punta
      shaderList[0].useShader();
      uniformModel = shaderList[0].getModelLocation();
      uniformProjection = shaderList[0].getProjectLocation();
      uniformView = shaderList[0].getViewLocation();
      uniformColor = shaderList[0].getColorLocation();

      model = glm::mat4(1.0);
      model = glm::translate(model, glm::vec3(0.2f, 0.9f, -4.85f));
      model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
      //model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
      model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
      glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
      glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
      glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
      color = glm::vec3(0.0f, 1.0f, 0.0f);
      glUniform3fv(uniformColor, 1, glm::value_ptr(color));
      meshList[1]->RenderMesh();

    // Renderizar la 2a fila (primera)
      shaderList[0].useShader();
      uniformModel = shaderList[0].getModelLocation();
      uniformProjection = shaderList[0].getProjectLocation();
      uniformView = shaderList[0].getViewLocation();
      uniformColor = shaderList[0].getColorLocation();

      model = glm::mat4(1.0);
      model = glm::translate(model, glm::vec3(0.45f, 0.12f, -4.42f));
      //model = glm::rotate(model, glm::radians(125.0f), glm::vec3(0.0f, 0.7f, 0.0f));
      //model = glm::rotate(model, glm::radians(5.0f), glm::vec3(0.0f, 0.0f, 1.0f));
      model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
      glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
      glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
      glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
      color = glm::vec3(0.0f, 1.0f, 0.0f);
      glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

```
//-----------------COLOR AMARILLO----------------------
    // Renderizar esquina de abajo
      shaderList[0].useShader();
      uniformModel = shaderList[0].getModelLocation();
      uniformProjection = shaderList[0].getProjectLocation();
      uniformView = shaderList[0].getViewLocation();
      uniformColor = shaderList[0].getColorLocation();

      model = glm::mat4(1.0);
      model = glm::translate(model, glm::vec3(0.7f, -0.8f, -4.12f));
      model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));
      model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
      glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
      glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
      glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
      color = glm::vec3(1.0f, 1.0f, 0.0f);
      glUniform3fv(uniformColor, 1, glm::value_ptr(color));
      meshList[1]->RenderMesh();

    // Renderizar 2da esquina de abajo
      shaderList[0].useShader();
      uniformModel = shaderList[0].getModelLocation();
      uniformProjection = shaderList[0].getProjectLocation();
      uniformView = shaderList[0].getViewLocation();
      uniformColor = shaderList[0].getColorLocation();

      model = glm::mat4(1.0);
      model = glm::translate(model, glm::vec3(-0.85f, -0.8f, -4.12f));
      model = glm::rotate(model, glm::radians(-5.0f), glm::vec3(1.0f, 0.0f, 0.0f));
      model = glm::scale(model, glm::vec3(0.5f, 0.5f, 0.5f));
      glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
      glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
      glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));
      color = glm::vec3(1.0f, 1.0f, 0.0f);
      glUniform3fv(uniformColor, 1, glm::value_ptr(color));
```

Problemas:

Ahora si me costo demasiado trabajo esta práctica, desde cómo me sentía de salud hasta enojarme por no poner bien las coordenadas, tuve problemas al rotar e insertar bien cada pirámide, la verdad que el azul ya lo dejé como pude porque ya no aguante.

3.- Conclusión:

a. Los ejercicios del reporte: La complejidad estuvo super elevada a mi parecer, cada vez se pone mucho más difícil esto.

b. Comentarios generales: Hubiera querido que se hiciera algún ejemplo mucho más especifico que solo las esferas, se podía tener o tomar el tiempo de explicar más a detalle cada figura y cada función.
c. Conclusión: Estuvo muy difícil, no estuve en mi mejor momento para realizar esta práctica pero se logró lo que se pudo lograr.